Федеральное государственное бюджетное образовательное учреждение высшего образования



«Московский государственный технический университет имени Н.Э. Баумана»
(национальный исследовательский университет)
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ____ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ_____

КАФЕДРА _____КОМПЬЮТЕРНЫЕ СИСТЕМЫ И СЕТИ (ИУ6)_____

О т ч е т

по домашнему заданию № 2

Название домашнего задания: Программирование с использованием библиотеки Qt

Дисциплина: Алгоритмизация и программирование

Студент гр. ИУ6-22Б _____ **М. Д. Акимов**
(Подпись, дата) (И.О. Фамилия)

Преподаватель _____ **О. А. Веселовская**
(Подпись, дата) (И.О. Фамилия)

Москва, 2026

Цель работы: освоение механизмов наследования и композиции в объектно-ориентированном программировании на языке C#, закрепление навыков проектирования иерархий классов, сокрытия полей, тестирования и протоколирования результатов.

ЧАСТЬ 1. НАСЛЕДОВАНИЕ

1.1. Постановка задачи

Разработать иерархию классов для предметной области «Шоколадные изделия».

- Базовый класс `ChocolateProduct` (шоколадное изделие) содержит поля: название, масса, энергетическая ценность на 100 грамм. Методы: процедура инициализации (конструктор), процедура вывода информации об объекте на экран (`DisplayInfo`), функция определения общей энергетической ценности изделия (`GetTotalEnergyValue`).
- Производный класс `ChocolateBar` (шоколадная плитка) добавляет поле «число долек в плитке» и переопределяет/расширяет методы: выводит информацию с числом долек, вычисляет энергетическую ценность всей плитки (`GetBarEnergyValue`) и одной дольки (`GetPieceEnergyValue`).

Все поля скрыты (`private`). Методы не содержат операций ввода-вывода, кроме `DisplayInfo`.

1.2. Диаграмма классов

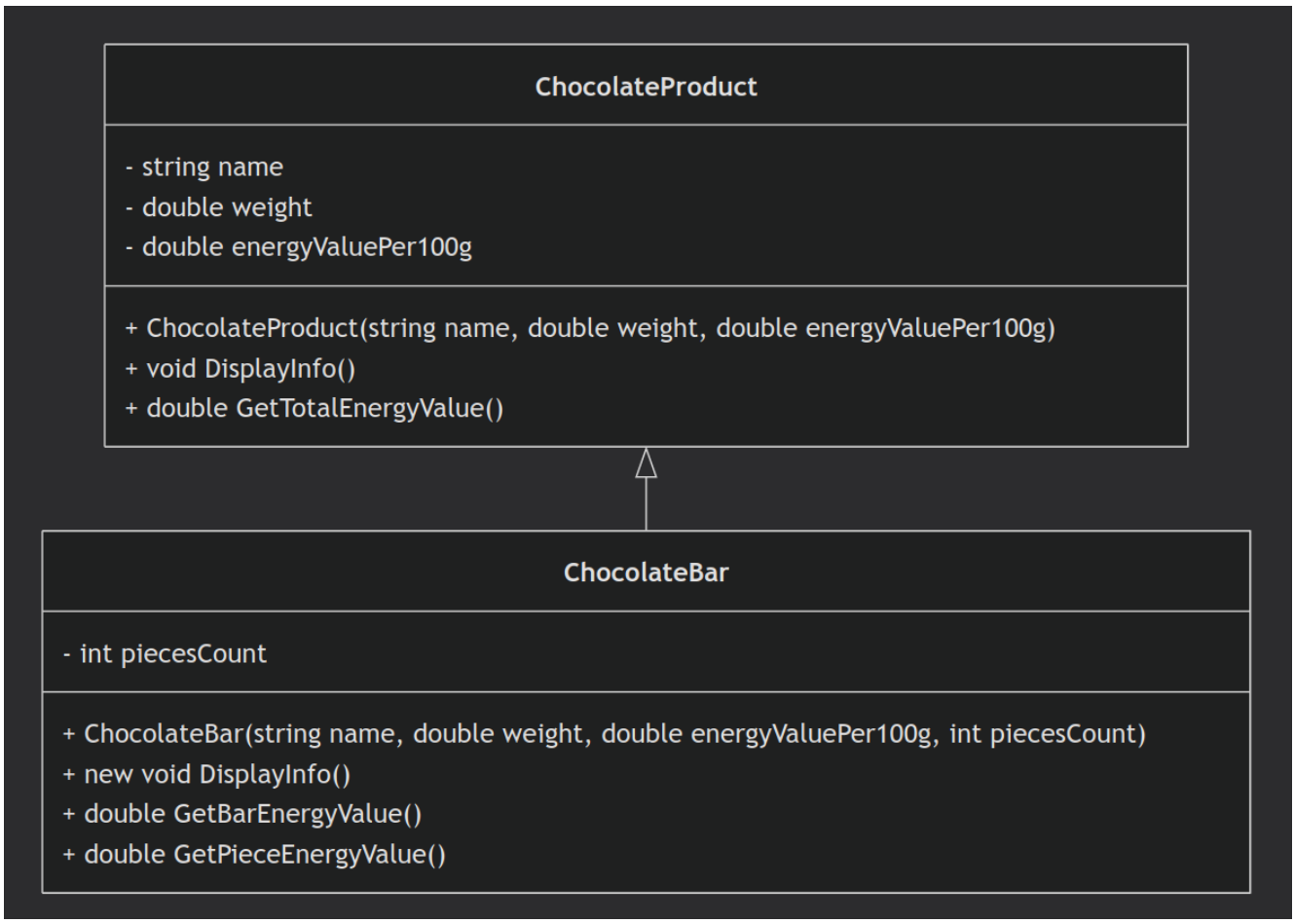


Рисунок 1 - Диаграмма классов

1.3. Объектная декомпозиция

При выполнении программы в методе `Main` создаются два объекта:

1. `product` типа `ChocolateProduct` с параметрами: название = «Горький шоколад», масса = 50 г, энергетическая ценность = 546 ккал/100 г.
2. `bar` типа `ChocolateBar` с параметрами: название = «Молочный шоколад», масса = 100 г, энергетическая ценность = 535 ккал/100 г, число долек = 8.

Объект `bar` является потомком `ChocolateProduct`, поэтому содержит все его поля и дополнительно поле `piecesCount`. Оба объекта существуют независимо, иллюстрируя иерархию «общее – частное».

1.4. Текст программы

```
using System;


class ChocolateProduct
{
    private string name;
    private double weight;
```

```
    private double energyValuePer100g;

    public ChocolateProduct(string name, double weight, double energyValuePer100g)
    {
        this.name = name;
        this.weight = weight;
        this.energyValuePer100g = energyValuePer100g;
    }

    public void DisplayInfo()
    {
        Console.WriteLine($"Название: {name}");
        Console.WriteLine($"Масса: {weight} г");
        Console.WriteLine($"Энергетическая ценность на 100 г: {energyValuePer100g} ккал");
    }

    public double GetTotalEnergyValue()
    {
        return (weight / 100) * energyValuePer100g;
    }
}

class ChocolateBar : ChocolateProduct
{
    private int piecesCount;

    public ChocolateBar(string name, double weight, double energyValuePer100g, int piecesCount)
        : base(name, weight, energyValuePer100g)
    {
        this.piecesCount = piecesCount;
    }

    new public void DisplayInfo()
```

```
    {
        base.DisplayInfo();
        Console.WriteLine($"Число долек: {piecesCount}");
    }

    public double GetBarEnergyValue()
    {
        return GetTotalEnergyValue();
    }

    public double GetPieceEnergyValue()
    {
        if (piecesCount <= 0) return 0;
        return GetBarEnergyValue() / piecesCount;
    }
}

class Program
{
    static void Main()
    {
        ChocolateProduct product = new ChocolateProduct("Горький шоколад", 50, 546);
        Console.WriteLine("Шоколадное изделие:");
        product.DisplayInfo();
        Console.WriteLine($"Общая энергетическая ценность: {product.GetTotalEnergyValue()}
ккал\n");

        ChocolateBar bar = new ChocolateBar("Молочный шоколад", 100, 535, 8);
        Console.WriteLine("Шоколадная плитка:");
        bar.DisplayInfo();
        Console.WriteLine($"Энергетическая ценность плитки: {bar.GetBarEnergyValue()} ккал");
        Console.WriteLine($"Энергетическая ценность одной дольки: {bar.GetPieceEnergyValue()}
ккал");
```

```
    }
}
```

1.5. Тестирование и результаты

Тест для `ChocolateProduct`**:**
Входные данные: название = «Горький шоколад», масса = 50 г, энерг. ценность 100 г = 546 ккал.
Ожидаемая общая энергетическая ценность = (50 / 100) * 546 = 273 ккал.
Вывод программы: «Общая энергетическая ценность: 273 ккал». Совпадает.

Тест для `ChocolateBar`**:**
Входные данные: название = «Молочный шоколад», масса = 100 г, энерг. ценность = 535 ккал, долек = 8.
Ожидаемая ценность плитки = (100/100)*535 = 535 ккал.
Ожидаемая ценность дольки = 535 / 8 = 66.875 ккал.
Вывод программы: «Энергетическая ценность плитки: 535 ккал», «Энергетическая ценность



Рисунок 2 - Результат работы программы

одной дольки: 66.875 ккал». Тест пройден.

ЧАСТЬ 2. КОМПОЗИЦИЯ

2.1. Постановка задачи

Разработать классы «Скаковая лошадь» и «Конюшня», используя механизм композиции.

- Класс `Horse` содержит поля: кличка (`string`), массив из 5 лучших рекордов (`double[]`). Методы: конструктор, `DisplayInfo()` (вывод информации), `GetAverageTime()` (возвращает среднее время).
- Класс `Stable` содержит массив объектов `Horse` и число лошадей (`count`). Методы: конструктор с параметром количества, `SetHorse(index, horse)` для добавления лошади, `DisplayInfo()` для вывода данных всех лошадей, `GetAverageTimeForStable()` – среднее время по всей конюшне.

Конюшня не наследует лошадь, а включает массив лошадей – это отношение «целое – часть» (композиция).

2.2. Диаграмма классов

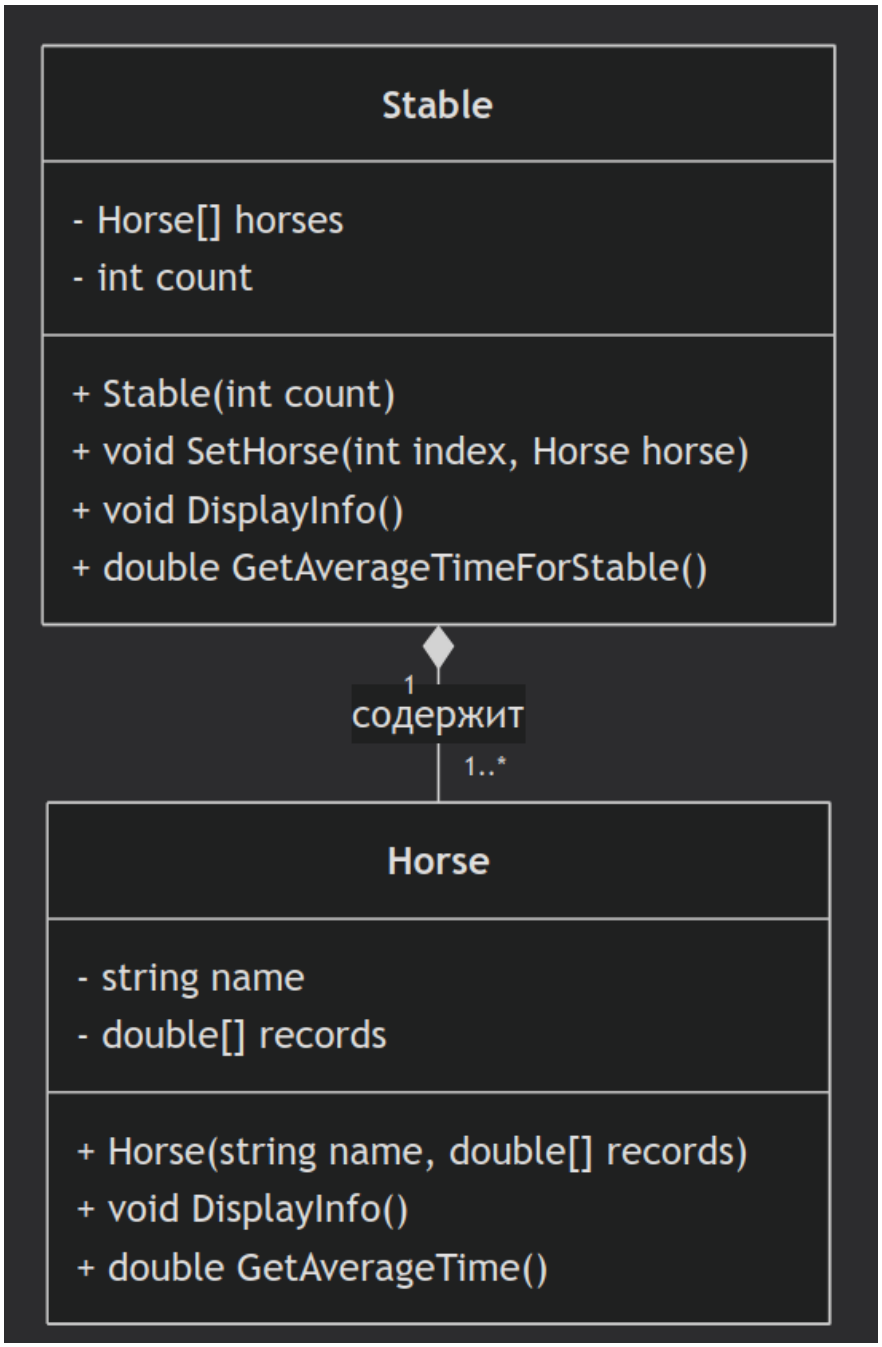


Рисунок 3 - Диаграмма классов

2.3. Объектная декомпозиция

В главной программе создаются три объекта `Horse`:

- `horse1` («Ветер», рекорды: 65.2, 64.8, 67.1, 66.0, 65.9)
- `horse2` («Молния», 68.5, 67.3, 69.0, 68.2, 67.8)
- `horse3` («Звезда», 62.1, 63.4, 61.9, 62.7, 63.0)

Затем создаётся объект `Stable` вместимостью 3. С помощью метода `SetHorse` в конюшню помещаются все три лошади. Конюшня становится владельцем этих объектов (композиция) – лошади не существуют отдельно от конюшни в рамках данной модели.

2.4. Текст программы

```
using System;

class Horse
{
    private string name;
    private double[] records;

    public Horse(string name, double[] records)
    {
        this.name = name;
        this.records = new double[5];
        for (int i = 0; i < 5; i++)
        {
            this.records[i] = records[i];
        }
    }

    public void DisplayInfo()
    {
        Console.WriteLine($"Кличка: {name}");
        Console.Write("Рекорды: ");
        for (int i = 0; i < 5; i++)
        {
            Console.Write($"{records[i]} ");
        }
        Console.WriteLine();
    }

    public double GetAverageTime()
    {
        double sum = 0;
        for (int i = 0; i < 5; i++)
```

```
        {
            sum += records[i];
        }
        return sum / 5;
    }
}

class Stable
{
    private Horse[] horses;
    private int count;

    public Stable(int count)
    {
        this.count = count;
        horses = new Horse[count];
    }

    public void SetHorse(int index, Horse horse)
    {
        if (index >= 0 && index < count)
        {
            horses[index] = horse;
        }
    }

    public void DisplayInfo()
    {
        Console.WriteLine($"Количество лошадей: {count}");
        for (int i = 0; i < count; i++)
        {
            Console.WriteLine($"Лошадь {i + 1}:");
            horses[i].DisplayInfo();
```

```
        }
    }

    public double GetAverageTimeForStable()
    {
        double total = 0;
        for (int i = 0; i < count; i++)
        {
            total += horses[i].GetAverageTime();
        }
        return total / count;
    }
}

class Program
{
    static void Main()
    {
        double[] records1 = { 65.2, 64.8, 67.1, 66.0, 65.9 };
        double[] records2 = { 68.5, 67.3, 69.0, 68.2, 67.8 };
        double[] records3 = { 62.1, 63.4, 61.9, 62.7, 63.0 };

        Horse horse1 = new Horse("Ветер", records1);
        Horse horse2 = new Horse("Молния", records2);
        Horse horse3 = new Horse("Звезда", records3);

        Stable stable = new Stable(3);
        stable.SetHorse(0, horse1);
        stable.SetHorse(1, horse2);
        stable.SetHorse(2, horse3);

        stable.DisplayInfo();
```

```
        Console.WriteLine($"Среднее время лошадей в конюшне:
{stable.GetAverageTimeForStable():F2}");
    }
}
```

2.5. Тестирование и результаты

Для лошади «Ветер» среднее время = (65.2+64.8+67.1+66.0+65.9)/5 = 65.8 с.
Для лошади «Молния» = (68.5+67.3+69.0+68.2+67.8)/5 = 68.16 с.
Для лошади «Звезда» = (62.1+63.4+61.9+62.7+63.0)/5 = 62.62 с.
Среднее по конюшне = (65.8 + 68.16 + 62.62) / 3 ≈ 65.53 с.

Вывод программы: «Среднее время лошадей в конюшне: 65.53». Совпадает с расчётом. Вывод



Рисунок 4 - Результат работы программы

информации о кличках и рекордах корректен.

ВЫВОДЫ

В ходе выполнения домашнего задания были изучены и применены два фундаментальных механизма построения классов в C#: наследование и композиция.

- В первой части показано, как создать иерархию «шоколадное изделие → шоколадная плитка» с переопределением и расширением функциональности.
- Во второй части реализовано отношение «целое – часть» между конюшней и лошадьми, где конюшня управляет массивом объектов `Horse`.

Все поля классов скрыты, доступ к ним осуществляется через методы. Тестирование подтвердило правильность вычислений. Приобретены навыки проектирования диаграмм классов и объектной декомпозиции для последующего документирования программ.